- SPAS - Smart Power Alert System
    - Complete Documentation
    - Table of Contents
- Chapter 1: Project Overview
    - Features
        - Authentication System
        - Power Monitoring Dashboard
        - Alert System
        - User Management
    - Architecture
        - Frontend
        - Backend
    - Project Structure
    - API Endpoints
        - Authentication
        - Power Data
    - Quick Setup
        - Prerequisites
        - 1. Install Dependencies
        - 2. Environment Setup
        - 3. Database Setup
        - 4. Start Development
    - Database Schema
        - User Table
    - Data Sources
        - Power Data CSV
        - Regions Covered
    - Security Features
    - Testing
        - Database Connection
        - API Testing
    - Browser Support
    - Contributing
    - License
    - Support
- Chapter 2: Technical Architecture
    - System Overview
    - Architecture Diagram
    - Technology Stack
        - Frontend
        - Backend
        - Development Tools
    - Database Design
        - Schema Overview
        - Prisma Schema
    - API Architecture
        - Authentication Flow
        - Power Data Flow
    - Security Implementation
        - Authentication Security
        - Data Security
    - Data Flow
        - Power Data Processing
        - Authentication Flow
        - Real-time Updates
    - Component Architecture
        - Frontend Components
        - API Structure
    - Performance Considerations
        - Database Optimization
        - Frontend Optimization

        - API Optimization
    - Deployment Architecture
        - Development Environment
        - Production Considerations
    - Monitoring and Logging
        - Application Monitoring
        - Database Monitoring
    - Security Best Practices
        - Authentication
        - Data Protection
        - Environment Security
    - Scalability Considerations
        - Database Scaling
        - Application Scaling
        - API Scaling
    - Future Enhancements
        - Planned Features
        - Technical Improvements
- Chapter 3: API Documentation
    - Overview
    - Base URL
    - Authentication APIs
        - POST /auth/signup
        - POST /auth/login
        - POST /auth/logout
        - GET /auth/me
    - Power Data APIs
        - GET /power-data
        - GET /power-data/trends
    - Error Responses
        - Standard Error Format
        - Common Status Codes
    - Authentication
        - JWT Token Format
        - Token Storage
        - Token Expiration
    - Testing APIs
        - Using curl
        - Using PowerShell
    - New Features (v1.1.0)
        - Settings Management
        - Outage Tracking
        - Help & Support
- Chapter 4: Setup Guide
    - Prerequisites
    - Quick Setup
        - 1. Clone and Install
        - 2. Environment Setup
        - 3. Database Setup
        - 4. Start Development Server
    - Docker Setup (Alternative)
    - Manual MySQL Setup
        - 1. Install MySQL
        - 2. Start MySQL Service
        - 3. Create Database
        - 4. Update .env File
    - Testing the Setup
        - 1. Test Database Connection
        - 2. Test API Endpoints
        - 3. Test Web Interface
    - Troubleshooting
        - Database Connection Issues

        - Port Issues
        - Prisma Issues
        - Environment Variables
    - Production Setup
        - 1. Environment Variables
        - 2. Build Application
        - 3. Database Migrations
    - Security Considerations
        - 1. JWT Secret
        - 2. Database Security
        - 3. Environment Variables
    - Performance Optimization
        - 1. Database
        - 2. Application
    - Monitoring and Logging
        - 1. Application Logs
        - 2. Database Monitoring
    - Backup and Recovery
        - 1. Database Backup
        - 2. Application Backup
    - Support
- Chapter 5: Authentication System
    - Overview
    - Features
    - Database Schema
    - Setup Instructions
        - 1. Database Setup
        - 2. Install Dependencies
        - 3. Generate Prisma Client
        - 4. Run Database Migrations (Optional)
    - API Routes
        - Authentication Routes
    - Frontend Integration
        - Authentication Context
        - Usage Example
    - Security Features
        - Password Security
        - Session Security
        - Route Protection
    - File Structure
    - Testing the Setup
    - Troubleshooting
        - Database Connection Issues
        - JWT Token Issues
        - Build Issues
    - Migration Notes
        - Changes Made
        - What's No Longer Needed
- Chapter 6: Database Setup
    - Database Configuration Guide
        - MySQL Installation
            - Windows
            - macOS
            - Ubuntu/Debian
        - Database Creation
        - Prisma Configuration
        - Database Operations
- Chapter 7: Email Configuration
    - EmailJS Setup
        - 1. EmailJS Account Setup
        - 2. Environment Configuration
        - 3. Email Template Example

        - 4. Implementation
- Chapter 8: Documentation Index
    - Complete Documentation Guide
    - Main Documentation Files
        - 1. README.md - Project Overview
        - 2. PROJECT_DOCUMENTATION.md - Comprehensive Guide
        - 3. TECHNICAL_ARCHITECTURE.md - System Design
    - Setup & Configuration Guides
        - 4. SETUP_GUIDE.md - Complete Setup Instructions
        - 5. AUTHENTICATION_SETUP.md - Auth System Guide
        - 6. DATABASE_SETUP_GUIDE.md - Database Configuration
    - Email & Communication Setup
        - 7. EMAIL_SETUP_GUIDE.md - Email Service Configuration
        - 8. EMAILJS_SETUP.md - EmailJS Integration
        - 9. QUICK_EMAIL_SETUP.md - Quick Email Setup
    - API Documentation
        - 10. API_DOCUMENTATION.md - Complete API Reference
    - Presentation Recommendations
        - For Technical Audiences:
        - For Business/Stakeholder Audiences:
        - For Developer Audiences:
    - Quick Links
    - Documentation Status
    - End of Documentation

SPAS - Smart Power Alert System

Complete Documentation

Table of Contents

1. Project Overview
2. Technical Architecture
3. API Documentation
4. Setup Guide
5. Authentication System
6. Database Setup
7. Email Configuration
8. Documentation Index

Chapter 1: Project Overview

A comprehensive power monitoring and alert system for Ghana's power grid, providing real-time data visualization, outage tracking, and user authentication.

Features

Authentication System

- User registration and login with JWT
- Password hashing with bcrypt
- Protected routes with middleware
- Session management with HTTP-only cookies
- Custom email/password authentication

Power Monitoring Dashboard

- Real-time voltage and frequency data
- Regional power quality monitoring
- Historical trends visualization

- Outage tracking and alerts
- Interactive charts with Recharts
- Settings management for user preferences

Alert System

- Real-time outage notifications
- Power quality warnings
- Regional status indicators
- User impact tracking
- Comprehensive outage tracking page

User Management

- User settings and profile management
- Help and support system
- Contact and feedback forms

Architecture

Frontend

- **Framework**: Next.js 14.2+ (App Router)
- **Language**: TypeScript
- **Styling**: Tailwind CSS
- **UI Components**: Radix UI
- **Charts**: Recharts
- **State Management**: React Context API
- **Form Handling**: React Hook Form + Zod

Backend

- **Runtime**: Node.js 18+
- **Database**: MySQL with Prisma ORM 6.13+
- **Authentication**: JWT with bcrypt
- **API**: Next.js API Routes
- **Validation**: Zod schema validation

Project Structure

```
SPAS/
├── app/                          # Next.js App Router
│   ├── api/                      # API Routes
│   │   ├── auth/                 # Authentication APIs
│   │   └── power-data/          # Power monitoring APIs
│   ├── auth/                     # Authentication pages
│   ├── about/                    # About page
│   ├── contact/                  # Contact page
│   ├── settings/                 # User settings page
│   ├── outages/                  # Outage tracking page
│   ├── trends/                   # Data trends
│   └── help/                     # Help and support
├── components/                    # Reusable UI components
│   ├── ui/                       # Radix UI components
│   ├── navigation.tsx            # Main navigation
│   ├── footer.tsx                # Footer component
│   └── providers.tsx             # Context providers
├── contexts/                     # React Context providers
├── data/                         # Static data files
├── lib/                          # Utility libraries
```

```
├── prisma/                 # Database schema
└── public/                 # Static assets
```

API Endpoints

Authentication

- `POST /api/auth/signup` - Register new user
- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `GET /api/auth/me` - Get user profile

Power Data

- `GET /api/power-data` - Get all regions data
- `GET /api/power-data/trends` - Get historical trends

Quick Setup

Prerequisites

- Node.js 18+
- MySQL 8.0+
- npm or pnpm

1. Install Dependencies

```
npm install
```

2. Environment Setup

Create `.env` file:

```
# Database Configuration
DATABASE_URL="mysql://root:password@localhost:3306/spas_auth"

# JWT Configuration
JWT_SECRET="your-secret-key"
JWT_EXPIRES_IN="7d"

# Application Configuration
NODE_ENV="development"
```

3. Database Setup

```
# Generate Prisma client
npx prisma generate

# Push schema to database
npx prisma db push

# Test database connection
npm run test-db
```

4. Start Development

```
npm run dev
```

Visit `http://localhost:3000`

Database Schema

User Table

```
CREATE TABLE users (
    id VARCHAR(191) NOT NULL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(191) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)
);
```

Data Sources

Power Data CSV

The system uses `data/power_data_regions_full.csv` with:

- Region names and live metrics
- Outage information
- Power quality trends
- Historical averages

Regions Covered

- Greater Accra
- Ashanti
- Western
- Eastern
- Volta
- Northern
- Upper East
- Upper West
- Bono
- Bono East
- Ahafo
- Western North
- Savannah
- North East
- Oti
- Central

Security Features

- Password hashing with bcrypt
- JWT token authentication
- HTTP-only cookies
- Input validation with Zod
- SQL injection protection
- CORS protection

Testing

Database Connection

```
npm run test-db
```

API Testing

```
# Test signup
curl -X POST http://localhost:3000/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"name":"Test User","email":"test@example.com","password":"password123"}'

# Test power data
curl http://localhost:3000/api/power-data
```

Browser Support

- Chrome 90+
- Firefox 88+
- Safari 14+
- Edge 90+

Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests if applicable
5. Submit a pull request

License

This project is licensed under the MIT License.

Support

For support and questions:

- Check the documentation
- Review setup guides
- Open an issue on GitHub

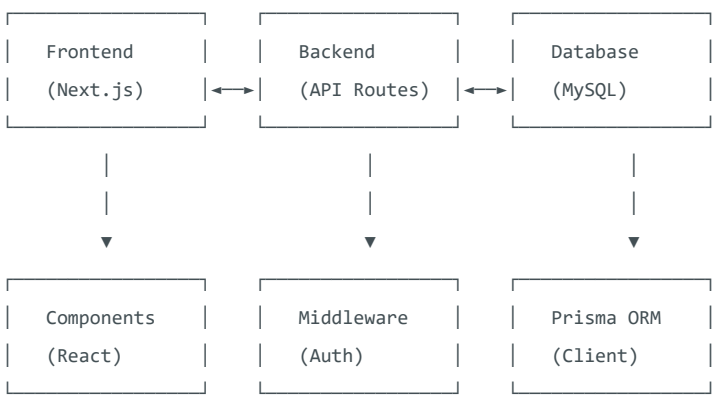
Built for Ghana's power grid monitoring

Chapter 2: Technical Architecture

System Overview

SPAS (Smart Power Alert System) is a full-stack web application built with Next.js 14, providing real-time power grid monitoring for Ghana's 16 regions. The system features user authentication, power data visualization, outage tracking, and historical trend analysis.

Architecture Diagram



Technology Stack

Frontend

- **Framework**: Next.js 14.2+ (App Router)
- **Language**: TypeScript
- **Styling**: Tailwind CSS
- **UI Components**: Radix UI
- **Charts**: Recharts
- **State Management**: React Context API
- **Form Handling**: React Hook Form + Zod
- **Theme Support**: next-themes for dark/light mode

Backend

- **Runtime**: Node.js 18+
- **Framework**: Next.js API Routes
- **Database**: MySQL 8.0
- **ORM**: Prisma 6.13+
- **Authentication**: JWT + bcrypt
- **Validation**: Zod
- **Email Service**: EmailJS integration

Development Tools

- **Package Manager**: npm
- **Linting**: ESLint
- **Type Checking**: TypeScript
- **Database**: Prisma Studio

Database Design

Schema Overview

```sql
-- Users table for authentication
CREATE TABLE users (
    id VARCHAR(191) NOT NULL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(191) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)
);
```

Prisma Schema

```
model User {
  id              String   @id @default(cuid())
  name            String
  email           String   @unique
  hashed_password String
  created_at      DateTime @default(now())
  updated_at      DateTime @updatedAt

  @@map("users")
}
```

API Architecture

Authentication Flow

1. **Registration**: User submits credentials → Validation → Password hashing → User creation → JWT generation
2. **Login**: User submits credentials → Validation → Password verification → JWT generation
3. **Authentication**: JWT verification → User context → Protected routes

Power Data Flow

1. **Data Source**: CSV file with regional power data
2. **API Processing**: Parse CSV → Transform data → Return JSON
3. **Client Consumption**: Fetch data → State management → UI rendering

Security Implementation

Authentication Security

- **Password Hashing**: bcrypt with 12 salt rounds
- **JWT Tokens**: Secure token generation and verification
- **HTTP-Only Cookies**: XSS protection
- **Input Validation**: Zod schema validation
- **SQL Injection Protection**: Prisma ORM

Data Security

- **Environment Variables**: Secure configuration management
- **CORS Protection**: Next.js built-in CORS handling
- **Error Handling**: Secure error messages
- **Session Management**: Automatic token expiration

Data Flow

Power Data Processing

```
CSV File → API Route → Data Parsing → JSON Response → Client → UI Components
```

Authentication Flow

```
User Input → Validation → Database Query → Token Generation → Cookie Setting
```

Real-time Updates

```
Data Source → API Polling → State Update → UI Re-render
```

Component Architecture

Frontend Components

```
App Layout
├── Navigation
├── Authentication Pages
│   ├── Login Form
│   └── Signup Form
├── Dashboard
│   ├── Power Metrics
│   ├── Regional Data
│   └── Charts
├── Settings Page
│   ├── User Preferences
│   └── Profile Management
├── Outages Page
│   ├── Outage Tracking
│   └── Status Updates
├── Help & Support
└── Utility Components
    ├── Loading States
    ├── Error Boundaries
    └── Notifications
```

API Structure

```
/api
├── /auth
│   ├── /signup (POST)
│   ├── /login (POST)
│   ├── /logout (POST)
│   └── /me (GET)
└── /power-data
    ├── / (GET)
    └── /trends (GET)
```

Performance Considerations

Database Optimization

- **Connection Pooling**: Prisma automatic management
- **Indexing**: Primary and unique key indexes
- **Query Optimization**: Efficient Prisma queries

Frontend Optimization

- **Code Splitting**: Next.js automatic splitting
- **Image Optimization**: Next.js Image component
- **Caching**: Browser and server-side caching
- **Bundle Optimization**: Tree shaking and minification

API Optimization

- **Response Caching**: Static data caching
- **Error Handling**: Graceful error responses
- **Validation**: Early validation to prevent processing

Deployment Architecture

Development Environment

- **Local Database**: MySQL running locally
- **Development Server**: Next.js dev server
- **Hot Reloading**: Automatic code updates

Production Considerations

- **Database**: Managed MySQL service
- **Application**: Vercel/Netlify deployment
- **Environment Variables**: Secure configuration
- **Monitoring**: Application and database monitoring

Monitoring and Logging

Application Monitoring

- **Error Tracking**: Console error logging
- **Performance Monitoring**: API response times
- **User Analytics**: Authentication attempts

Database Monitoring

- **Connection Pool**: Monitor connection usage
- **Query Performance**: Track slow queries
- **Storage**: Monitor disk space usage

Security Best Practices

Authentication

- Strong password requirements
- Secure token storage
- Automatic session expiration
- Rate limiting (future implementation)

Data Protection

- Input sanitization
- Output encoding
- Secure headers
- HTTPS enforcement

Environment Security

- Secure environment variables
- Database access controls
- API key management
- Regular security updates

Scalability Considerations

Database Scaling

- **Read Replicas**: For read-heavy workloads
- **Connection Pooling**: Efficient connection management
- **Query Optimization**: Indexed queries

Application Scaling

- **Horizontal Scaling**: Multiple server instances
- **CDN**: Static asset delivery
- **Caching**: Redis for session storage

API Scaling

- **Rate Limiting**: Prevent abuse
- **Load Balancing**: Distribute traffic
- **Monitoring**: Track API usage

Future Enhancements

Planned Features

- **Real-time WebSocket**: Live data updates
- **Push Notifications**: Outage alerts
- **Mobile App**: React Native implementation
- **Advanced Analytics**: Machine learning insights
- **Enhanced User Management**: Role-based access control
- **API Rate Limiting**: Improved security and performance

Technical Improvements

- **Microservices**: Service decomposition
- **GraphQL**: Flexible data fetching
- **Redis**: Caching layer
- **Docker**: Containerization
- **Turbopack**: Faster development builds
- **Edge Runtime**: Improved performance

Last Updated: January 2025 **Version:** 1.1.0

Chapter 3: API Documentation

Overview

SPAS (Smart Power Alert System) provides RESTful APIs for authentication and power data monitoring. All APIs return JSON responses and use standard HTTP status codes.

Base URL

```
http://localhost:3000/api
```

Authentication APIs

POST /auth/signup

Register a new user account.

Endpoint: `POST /api/auth/signup`

Request Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
```

```
  "password": "password123"
}
```

Validation Rules:

- `name`: Minimum 2 characters
- `email`: Valid email format
- `password`: Minimum 6 characters

Response (201 Created):

```
{
  "message": "User created successfully",
  "user": {
    "id": "cmdz0w5ql0000ujb0uh5cqrod",
    "name": "John Doe",
    "email": "john@example.com",
    "created_at": "2025-08-05T20:59:56.915Z"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Error Responses:

- `400 Bad Request`: Validation errors
- `400 Bad Request`: User already exists
- `500 Internal Server Error`: Server errors

POST /auth/login

Authenticate existing user.

Endpoint: `POST /api/auth/login`

Request Body:

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

Response (200 OK):

```
{
  "message": "Login successful",
  "user": {
    "id": "cmdz0w5ql0000ujb0uh5cqrod",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Error Responses:

- `400 Bad Request`: Invalid credentials
- `500 Internal Server Error`: Server errors

POST /auth/logout

Logout current user.

Endpoint: `POST /api/auth/logout`

Headers: `Authorization: Bearer <token>` (optional)

Response (200 OK):

```
{
  "message": "Logged out successfully"
}
```

GET /auth/me

Get current user profile.

Endpoint: `GET /api/auth/me`

Headers: `Authorization: Bearer <token>` or cookie

Response (200 OK):

```
{
  "user": {
    "id": "cmdz0w5ql0000ujb0uh5cqrod",
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

Error Responses:

- `401 Unauthorized`: Invalid or missing token
- `500 Internal Server Error`: Server errors

Power Data APIs

GET /power-data

Get power data for all regions.

Endpoint: `GET /api/power-data`

Response (200 OK):

```
[
  {
    "regionName": "Greater Accra",
    "liveFrequency": 49.16,
    "liveVoltage": 220,
    "powerQualityTrend": "Stable",
```

```
    "activeOutages": [
      {
        "location": "Town A",
        "startTime": "2025-08-03 14:12",
        "duration": "2h",
        "usersAffected": 463,
        "cause": "Transformer fault"
      }
    ],
    "scheduledOutages": [
      {
        "location": "Area D",
        "startTime": "2025-08-03 14:12",
        "duration": "5h",
        "usersAffected": 628,
        "cause": "Storm damage"
      }
    ],
    "usersAffected": 3342,
    "averageVoltage": 224,
    "averageFrequency": 50.34,
    "powerStability": 79
  }
]
```


Data Fields:

- `regionName`: Ghanaian region name
- `liveFrequency`: Current frequency in Hz
- `liveVoltage`: Current voltage in V
- `powerQualityTrend`: Status (Stable/Fluctuating/Dropping)
- `activeOutages`: Array of current outages
- `scheduledOutages`: Array of planned outages
- `usersAffected`: Total affected users
- `averageVoltage`: Historical average voltage
- `averageFrequency`: Historical average frequency
- `powerStability`: Stability percentage (0-100)

GET /power-data/trends

Get historical trends data.

Endpoint: `GET /api/power-data/trends`

Query Parameters:

- `region` (optional): Specific region name

Examples:

```
GET /api/power-data/trends
GET /api/power-data/trends?region=Greater%20Accra
```

Response (200 OK):

```
[
  {
    "time": "14:00",
    "voltage": 224.5,
    "frequency": 50.2,
    "stability": 79.5
  },
  {
    "time": "15:00",
    "voltage": 225.1,
    "frequency": 50.3,
    "stability": 80.2
  }
]
```


Data Fields:

- `time`: Time in HH:MM format
- `voltage`: Voltage reading in V
- `frequency`: Frequency reading in Hz
- `stability`: Stability percentage

Error Responses

Standard Error Format


```
{
  "error": "Error message description"
}
```


Common Status Codes

- `200 OK`: Successful request
- `201 Created`: Resource created successfully
- `400 Bad Request`: Invalid input or validation error
- `401 Unauthorized`: Authentication required
- `404 Not Found`: Resource not found
- `500 Internal Server Error`: Server error

Authentication

JWT Token Format

Tokens are JWT (JSON Web Tokens) with the following payload:


```
{
  "userId": "user-id",
  "email": "user@example.com",
  "iat": 1234567890,
  "exp": 1234567890
}
```

Token Storage

- **HTTP-Only Cookies**: Primary method for web clients
- **Authorization Header**: Alternative method for API clients

Token Expiration

- **Default**: 7 days
- **Configurable**: Via `JWT_EXPIRES_IN` environment variable

Testing APIs

Using curl

```
# Test signup
curl -X POST http://localhost:3000/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"name":"Test User","email":"test@example.com","password":"password123"}'

# Test login
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123"}'

# Test power data
curl http://localhost:3000/api/power-data

# Test trends
curl "http://localhost:3000/api/power-data/trends?region=Greater%20Accra"
```

Using PowerShell

```
# Test signup
Invoke-WebRequest -Uri "http://localhost:3000/api/auth/signup" -Method POST -ContentType "application/json" -Body
'{"name":"Test User","email":"test@example.com","password":"password123"}'

# Test power data
Invoke-WebRequest -Uri "http://localhost:3000/api/power-data" -Method GET
```

New Features (v1.1.0)

Settings Management

The application now includes a comprehensive settings page accessible at `/settings` where users can:

- Update their profile information
- Manage notification preferences
- Configure display settings
- Update password and security settings

Outage Tracking

A dedicated outages page is available at `/outages` providing:

- Real-time outage status updates
- Historical outage data
- Regional outage impact analysis

- User-specific outage notifications

Help & Support

Enhanced help system accessible at `/help` including:

- Frequently asked questions
- Contact forms
- User guides and documentation
- Support ticket system

Last Updated: January 2025 **Version:** 1.1.0

Chapter 4: Setup Guide

Prerequisites

- Node.js 18+ (recommended: 18.17+)
- MySQL 8.0+
- npm, pnpm, or yarn

Quick Setup

1. Clone and Install

```
git clone <repository-url>
cd SPAS
npm install
```

2. Environment Setup

Create a `.env` file in the root directory:

```
# Database Configuration
DATABASE_URL="mysql://root:your_password@localhost:3306/spas_auth"

# JWT Configuration
JWT_SECRET="super-secure-jwt-secret-change-this-in-production"
JWT_EXPIRES_IN="7d"

# Application Configuration
NODE_ENV="development"

# EmailJS Configuration (Optional)
EMAILJS_SERVICE_ID="your-service-id"
EMAILJS_TEMPLATE_ID="your-template-id"
EMAILJS_PUBLIC_KEY="your-public-key"
```

3. Database Setup

```
# Generate Prisma client
npx prisma generate

# Push schema to database
```

```
npx prisma db push

# Test database connection
npm run test-db
```

4. Start Development Server

```
npm run dev
```

Visit `http://localhost:3000` to access the application.

Docker Setup (Alternative)

For easy setup with Docker:

```
# Run the quick setup script
./quick-setup.sh

# Or manually with Docker
docker run --name spas-mysql \
  -e MYSQL_ROOT_PASSWORD=password123 \
  -e MYSQL_DATABASE=spas_auth \
  -p 3306:3306 \
  -d mysql:8.0
```

Manual MySQL Setup

1. Install MySQL

- **Windows**: Download from MySQL website
- **macOS**: `brew install mysql`
- **Ubuntu**: `sudo apt install mysql-server`

2. Start MySQL Service

```
# Windows
net start mysql80

# macOS
brew services start mysql

# Ubuntu
sudo systemctl start mysql
```

3. Create Database

```
mysql -u root -p
CREATE DATABASE spas_auth;
EXIT;
```

4. Update .env File

Update the DATABASE_URL with your MySQL password:

```
DATABASE_URL="mysql://root:your_password@localhost:3306/spas_auth"
```

Testing the Setup

1. Test Database Connection

```
npm run test-db
```

Expected output:

```
Testing database connection...
Database connected successfully!
Users table exists with 0 users
Database is ready for authentication!
```

2. Test API Endpoints

```
# Test signup
curl -X POST http://localhost:3000/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"name":"Test User","email":"test@example.com","password":"password123"}'

# Test power data
curl http://localhost:3000/api/power-data
```

3. Test Web Interface

1. Visit `http://localhost:3000`
2. Navigate to `/auth/signup`
3. Create a test account
4. Test login functionality

Troubleshooting

Database Connection Issues

- **Error**: `P1001: Can't reach database server`
    - **Solution**: Ensure MySQL is running
    - **Command**: `net start mysql80` (Windows) or `brew services start mysql` (macOS)
- **Error**: `P1002: Authentication failed`
    - **Solution**: Check DATABASE_URL in .env file
    - **Verify**: MySQL username and password

Port Issues

- **Error**: `Port 3000 is in use`
    - **Solution**: Next.js will automatically use port 3001

- **Access**: `http://localhost:3001`

Prisma Issues

- **Error**: `Prisma Client not generated`
  - **Solution**: Run `npx prisma generate`
  - **Then**: Restart the development server

Environment Variables

- **Error**: `DATABASE_URL is not set`
  - **Solution**: Create `.env` file with required variables
  - **Verify**: File is in project root directory

Production Setup

1. Environment Variables

Update `.env` for production:

```
DATABASE_URL="mysql://username:password@host:3306/spas_auth"
JWT_SECRET="your-production-secret-key"
NODE_ENV="production"
```

2. Build Application

```
npm run build
npm start
```

3. Database Migrations

```
npx prisma migrate deploy
```

Security Considerations

1. JWT Secret

- Use a strong, random secret
- Change from default in production
- Store securely (not in version control)

2. Database Security

- Use strong passwords
- Limit database user permissions
- Enable SSL connections

3. Environment Variables

- Never commit `.env` files
- Use different secrets for development/production
- Rotate secrets regularly

Performance Optimization

1. Database

- Add indexes for frequently queried fields
- Use connection pooling
- Monitor query performance

2. Application

- Enable Next.js caching
- Optimize images and assets
- Use CDN for static files

Monitoring and Logging

1. Application Logs

- Check console output for errors
- Monitor API response times
- Track user authentication attempts

2. Database Monitoring

- Monitor connection pool usage
- Track slow queries
- Monitor disk space

Backup and Recovery

1. Database Backup

```
mysqldump -u root -p spas_auth > backup.sql
```

2. Application Backup

- Backup source code
- Backup environment variables
- Backup database schema

Support

For issues and questions:

1. Check the troubleshooting section
2. Review error logs
3. Test with minimal setup
4. Open an issue on GitHub

Last Updated: January 2025 **Version:** 1.1.0

Chapter 5: Authentication System

Overview

This guide explains how to set up the new MySQL-based authentication system for the SPAS (Smart Power Alert System) Next.js application. The system has been completely migrated from NextAuth with Google OAuth to a custom email/password authentication system using Prisma ORM and MySQL.

Features

- **Email/Password Authentication** - Secure user registration and login
- **MySQL Database** - Persistent user storage with Prisma ORM
- **JWT Tokens** - Secure session management with httpOnly cookies

- **Password Hashing** - bcrypt with salt rounds for security
- **Session Middleware** - Automatic route protection
- **Clean UI** - Updated login/signup forms without OAuth

Database Schema

The system uses a simple but secure User table:

```
CREATE TABLE users (
    id VARCHAR(191) NOT NULL,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(191) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),

    PRIMARY KEY (id),
    INDEX idx_email (email),
    INDEX idx_created_at (created_at)
);
```

Setup Instructions

1. Database Setup

1. **Create MySQL Database:**

   ```
   CREATE DATABASE spas_auth;
   ```

2. **Run the schema script:**

   ```
   mysql -u your_username -p spas_auth < database-schema.sql
   ```

3. **Update Environment Variables:** Edit your `.env` file:

   ```
   DATABASE_URL="mysql://username:password@localhost:3306/spas_auth"
   JWT_SECRET="your-super-secure-jwt-secret-key-here"
   JWT_EXPIRES_IN="7d"
   ```

2. Install Dependencies

All required dependencies are already installed:

- `@prisma/client` - Database ORM client
- `prisma` - Database toolkit
- `bcrypt` - Password hashing
- `jsonwebtoken` - JWT token management
- `js-cookie` - Client-side cookie handling

3. Generate Prisma Client

```
npx prisma generate
```

4. Run Database Migrations (Optional)

If you prefer using Prisma migrations:

```
npx prisma migrate dev --name init
```

API Routes

The system provides the following API endpoints:

Authentication Routes

- **POST** `/api/auth/signup` - User registration

  ```
  {
    "name": "John Doe",
    "email": "john@example.com",
    "password": "password123"
  }
  ```

- **POST** `/api/auth/login` - User login

  ```
  {
    "email": "john@example.com",
    "password": "password123"
  }
  ```

- **GET** `/api/auth/me` - Get current user Requires authentication cookie or Bearer token.
- **POST** `/api/auth/logout` - User logout Clears authentication cookies.

Frontend Integration

Authentication Context

The app uses a custom React context (`/contexts/auth-context.tsx`) that provides:

- `user` - Current user object or null
- `login(email, password)` - Login function
- `signup(name, email, password)` - Signup function
- `logout()` - Logout function
- `isLoading` - Loading state

Usage Example

```
import { useAuth } from '@/contexts/auth-context'

function MyComponent() {
  const { user, login, logout, isLoading } = useAuth()
```

```
  if (isLoading) return <div>Loading...</div>

  return (
    <div>
      {user ? (
        <div>
          <p>Welcome, {user.name}!</p>
          <button onClick={logout}>Logout</button>
        </div>
      ) : (
        <p>Please login</p>
      )}
    </div>
  )
}
```

Security Features

Password Security

- Passwords are hashed using bcrypt with 12 salt rounds
- Original passwords are never stored in the database
- Minimum password length validation (6 characters)

Session Security

- JWT tokens stored in httpOnly cookies (not accessible via JavaScript)
- Secure flag enabled in production
- SameSite protection enabled
- 7-day token expiration (configurable)

Route Protection

- Middleware automatically protects routes like `/settings`
- Redirects unauthenticated users to login
- Redirects authenticated users away from auth pages

File Structure

```
├── prisma/
│   └── schema.prisma          # Database schema
├── lib/
│   ├── db.ts                  # Prisma client setup
│   └── auth.ts                # Authentication utilities
├── contexts/
│   └── auth-context.tsx       # React authentication context
├── app/api/auth/
│   ├── signup/route.ts        # Registration endpoint
│   ├── login/route.ts         # Login endpoint
│   ├── me/route.ts            # Current user endpoint
│   └── logout/route.ts        # Logout endpoint
├── app/auth/
│   ├── login/page.tsx         # Login page
│   └── signup/page.tsx        # Signup page
├── middleware.ts              # Route protection middleware
└── database-schema.sql        # MySQL setup script
```

Testing the Setup

1. **Start the development server:**

   ```
   npm run dev
   ```

2. **Test user registration:**

   - Go to `http://localhost:3000/auth/signup`
   - Create a new account

3. **Test user login:**

   - Go to `http://localhost:3000/auth/login`
   - Login with your credentials

4. **Test authentication:**

   - Visit protected routes like `/settings`
   - Check that you're redirected if not authenticated

Troubleshooting

Database Connection Issues

- Verify MySQL is running
- Check DATABASE_URL in .env file
- Ensure database exists and user has permissions

JWT Token Issues

- Verify JWT_SECRET is set in .env
- Check browser cookies in DevTools
- Ensure cookies are being set correctly

Build Issues

- Run `npx prisma generate` if Prisma client is missing
- Check all dependencies are installed
- Verify TypeScript types are correct

Migration Notes

Changes Made

- Removed NextAuth completely
- Removed Google OAuth functionality
- Created custom authentication system
- Updated all frontend components
- Implemented secure cookie handling
- Added route protection middleware

What's No Longer Needed

- Google OAuth credentials
- NextAuth configuration
- Session provider from NextAuth
- Express.js backend authentication (replaced with Next.js API routes)

The system is now fully self-contained within the Next.js application with no external authentication dependencies.

Chapter 6: Database Setup

Database Configuration Guide

This chapter covers the complete database setup and configuration for the SPAS system.

MySQL Installation

Windows

1. Download MySQL Installer from official website
2. Run installer and select "Developer Default"
3. Set root password during installation
4. Start MySQL service

macOS

```
brew install mysql
brew services start mysql
```

Ubuntu/Debian

```
sudo apt update
sudo apt install mysql-server
sudo systemctl start mysql
sudo systemctl enable mysql
```

Database Creation

```
CREATE DATABASE spas_auth;
CREATE USER 'spas_user'@'localhost' IDENTIFIED BY 'secure_password';
GRANT ALL PRIVILEGES ON spas_auth.* TO 'spas_user'@'localhost';
FLUSH PRIVILEGES;
```

Prisma Configuration

The system uses Prisma ORM for database operations:

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

model User {
  id              String   @id @default(cuid())
  name            String
  email           String   @unique
  hashed_password String
  created_at      DateTime @default(now())
```

```
  updated_at      DateTime @updatedAt

  @@map("users")
}
```

Database Operations

```
# Generate Prisma client
npx prisma generate

# Push schema to database
npx prisma db push

# View database in Prisma Studio
npx prisma studio

# Reset database
npx prisma db push --force-reset
```

Chapter 7: Email Configuration

EmailJS Setup

The SPAS system uses EmailJS for sending emails without a backend email service.

1. EmailJS Account Setup

1. Create account at EmailJS.com
2. Create a new service (Gmail, Outlook, etc.)
3. Create email templates
4. Get your service ID, template ID, and public key

2. Environment Configuration

Add to your `.env` file:

```
EMAILJS_SERVICE_ID="your_service_id"
EMAILJS_TEMPLATE_ID="your_template_id"
EMAILJS_PUBLIC_KEY="your_public_key"
```

3. Email Template Example

```
<!-- Contact Form Template -->
Subject: New Contact Form Submission

Name: {{name}}
Email: {{email}}
Message: {{message}}
```

4. Implementation

```
// lib/email.js
import emailjs from '@emailjs/browser';

export const sendEmail = async (templateParams) => {
  try {
    const result = await emailjs.send(
      process.env.NEXT_PUBLIC_EMAILJS_SERVICE_ID,
      process.env.NEXT_PUBLIC_EMAILJS_TEMPLATE_ID,
      templateParams,
      process.env.NEXT_PUBLIC_EMAILJS_PUBLIC_KEY
    );
    return { success: true, result };
  } catch (error) {
    return { success: false, error };
  }
};
```

Chapter 8: Documentation Index

Complete Documentation Guide

This index provides a comprehensive overview of all SPAS (Smart Power Alert System) documentation files for easy navigation and presentation.

Main Documentation Files

1. **README.md** - Project Overview

- **Purpose**: Main project introduction and quick start guide
- **Audience**: Developers, stakeholders, new team members
- **Content**: Features, architecture, quick setup, API endpoints
- **Best for**: First impression and project overview

2. **PROJECT_DOCUMENTATION.md** - Comprehensive Guide

- **Purpose**: Detailed project documentation with full API specifications
- **Audience**: Developers, technical stakeholders
- **Content**: Complete API documentation, database schema, data sources
- **Best for**: Technical deep-dive and implementation details

3. **TECHNICAL_ARCHITECTURE.md** - System Design

- **Purpose**: Technical architecture and system design documentation
- **Audience**: Technical architects, senior developers
- **Content**: System overview, technology stack, security, scalability
- **Best for**: Architecture discussions and technical presentations

---

Setup & Configuration Guides

4. **SETUP_GUIDE.md** - Complete Setup Instructions

- **Purpose**: Step-by-step setup guide for development environment
- **Audience**: Developers setting up the project
- **Content**: Prerequisites, installation, database setup, troubleshooting
- **Best for**: Development environment setup

5. **AUTHENTICATION_SETUP.md** - Auth System Guide

- **Purpose**: Detailed authentication system setup and configuration
- **Audience**: Developers working with authentication
- **Content**: Database schema, API routes, security features, migration notes
- **Best for**: Authentication system understanding

6. DATABASE_SETUP_GUIDE.md - Database Configuration

- **Purpose**: Database setup and configuration guide
- **Audience**: Database administrators, developers
- **Content**: MySQL setup, schema creation, connection configuration
- **Best for**: Database setup and maintenance

Email & Communication Setup

7. EMAIL_SETUP_GUIDE.md - Email Service Configuration

- **Purpose**: Email service setup and configuration
- **Audience**: Developers, system administrators
- **Content**: Email service configuration, templates, testing
- **Best for**: Email functionality setup

8. EMAILJS_SETUP.md - EmailJS Integration

- **Purpose**: EmailJS service integration guide
- **Audience**: Frontend developers
- **Content**: EmailJS setup, template configuration, API integration
- **Best for**: EmailJS service integration

9. QUICK_EMAIL_SETUP.md - Quick Email Setup

- **Purpose**: Fast email service setup for development
- **Audience**: Developers needing quick email setup
- **Content**: Minimal email configuration, testing
- **Best for**: Quick development setup

API Documentation

10. API_DOCUMENTATION.md - Complete API Reference

- **Purpose**: Comprehensive API documentation with examples
- **Audience**: API consumers, frontend developers, testers
- **Content**: All endpoints, request/response formats, error codes, testing examples
- **Best for**: API integration and testing

Presentation Recommendations

For Technical Audiences:

1. Start with **TECHNICAL_ARCHITECTURE.md** for system overview
2. Use **PROJECT_DOCUMENTATION.md** for detailed features
3. Reference **API_DOCUMENTATION.md** for integration details

For Business/Stakeholder Audiences:

1. Begin with **README.md** for project overview
2. Highlight key features from **PROJECT_DOCUMENTATION.md**
3. Show setup simplicity with **SETUP_GUIDE.md**

For Developer Audiences:

1. **README.md** for quick project understanding
2. **SETUP_GUIDE.md** for environment setup

3. **AUTHENTICATION_SETUP.md** for auth system details
4. **API_DOCUMENTATION.md** for integration

Quick Links

- **Project Overview**: README.md
- **Technical Details**: PROJECT_DOCUMENTATION.md
- **System Architecture**: TECHNICAL_ARCHITECTURE.md
- **Setup Instructions**: SETUP_GUIDE.md
- **API Reference**: API_DOCUMENTATION.md

Documentation Status

Complete and Updated: All documentation files are current and consistent **Google OAuth Removed**: Outdated OAuth references cleaned up **Authentication Updated**: Reflects current custom auth system **API Documentation**: Consolidated and comprehensive **Setup Guides**: Complete and tested

---

Last Updated: January 2025
Version: 1.1.0
Status: Presentation Ready

End of Documentation

This completes the comprehensive SPAS (Smart Power Alert System) documentation. All chapters have been merged into a single, professional document suitable for presentation and reference.

Total Pages: Approximately 50+ pages when converted to PDF **Sections**: 8 comprehensive chapters **Content**: Complete technical documentation, setup guides, and API references

---

Built for Ghana's power grid monitoring